

```

1 import pandas as pd
2 import numpy as np
3 import os
4
5 # STEP 1 – LOAD DATASET
6
7 print("=" * 55)
8 print("STEP 1 – LOADING DATASET")
9 print("=" * 55)
10
11 df = pd.read_csv(r"C:\Users\Harikesh\OneDrive\Desktop\Sales_Dashboard.csv")
12
13 print(f"Total Rows      : {df.shape[0]}")
14 print(f"Total Columns : {df.shape[1]}")
15 print(f"\nColumn Names  : \n{list(df.columns)}")
16 print(f"\nFirst 3 Rows   : \n{df.head(3)}")
17
18
19 # STEP 2 – BASIC INFO & DATA TYPES
20
21 print("\n" + "=" * 55)
22 print("STEP 2 – DATA TYPES & BASIC INFO")
23 print("=" * 55)
24
25 print(df.info())
26 print(f"\nStatistical Summary: \n{df.describe()}")
27
28
29 # STEP 3 – CHECK MISSING VALUES
30
31 print("\n" + "=" * 55)
32 print("STEP 3 – MISSING VALUES CHECK")
33 print("=" * 55)
34
35 missing = df.isnull().sum()
36 missing_pct = (df.isnull().sum() / len(df)) * 100
37
38 missing_df = pd.DataFrame({
39     'Missing Count' : missing,
40     'Missing Percent': missing_pct
41 })
42 print(missing_df)
43
44 # Your dataset has 0 missing values – no action needed
45 if missing.sum() == 0:
46     print("\n No missing values found – dataset is clean!")
47 else:
48     # Fill missing numeric columns with median
49     num_cols = df.select_dtypes(include='number').columns
50     df[num_cols] = df[num_cols].fillna(df[num_cols].median())
51
52     # Fill missing string columns with 'Unknown'
53     str_cols = df.select_dtypes(include='object').columns
54     df[str_cols] = df[str_cols].fillna('Unknown')
55     print("\n Missing values filled!")
56
57
58 # STEP 4 – CHECK & REMOVE DUPLICATES
59
60 print("\n" + "=" * 55)

```

```

61 print("STEP 4 – DUPLICATE CHECK")
62 print("=" * 55)
63
64 before = len(df)
65 print(f"Rows before : {before}")
66 print(f"Duplicates : {df.duplicated().sum()}")
67
68 df = df.drop_duplicates()
69 print(f"Rows after : {len(df)}")
70
71 # Your dataset has 0 duplicates
72 if len(df) == before:
73     print(" No duplicates found!")
74 else:
75     print(f" Removed {before - len(df)} duplicate rows!")
76
77
78 # STEP 5 – FIX DATA TYPES
79
80 print("\n" + "=" * 55)
81 print("STEP 5 – FIX DATA TYPES")
82 print("=" * 55)
83
84 # Convert order_date → datetime (was string/object)
85 df['order_date'] = pd.to_datetime(df['order_date'])
86 print(f"order_date dtype : {df['order_date'].dtype}")
87
88 # Strip whitespace from all string columns
89 str_cols = df.select_dtypes(include='object').columns
90 for col in str_cols:
91     df[col] = df[col].str.strip()
92
93 print(" Data types fixed!")
94 print(f"\nDate Range : {df['order_date'].min().date()} to "
95       f"{df['order_date'].max().date()}")
96
97
98 # STEP 6 – VALIDATE COLUMN VALUES
99
100 print("\n" + "=" * 55)
101 print("STEP 6 – VALIDATE UNIQUE VALUES")
102 print("=" * 55)
103
104 print(f"Status values : {df['status'].unique()}")
105 print(f"Region values : {df['region'].unique()}")
106 print(f"Category values : {df['category'].unique()}")
107 print(f"\nStatus counts:\n{df['status'].value_counts()}")
108 print(f"\nRegion counts:\n{df['region'].value_counts()}")
109 print(f"\nCategory counts:\n{df['category'].value_counts()}")
110
111
112 # STEP 7 – FEATURE ENGINEERING (Add New Useful Columns)
113
114 print("\n" + "=" * 55)
115 print("STEP 7 – FEATURE ENGINEERING")
116 print("=" * 55)
117
118 # — Date Features —————
119 df['order_day_name'] = df['order_date'].dt.day_name()
120 df['order_quarter'] = df['order_date'].dt.quarter

```

```

121 df['order_week']          = df['order_date'].dt.isocalendar().week.astype(int)
122
123 # — Financial Features —————
124 df['discount_amount'] = round(
125     df['unit_price'] * df['quantity'] * df['discount_pct'] / 100, 2)
126
127 df['gross_amount']      = df['unit_price'] * df['quantity']
128
129 df['profit']            = round(df['sales_amount'] * 0.30, 2)
130
131 df['profit_margin_pct'] = round(
132     (df['profit'] / df['sales_amount']) * 100, 2)
133
134 # — Order Size Category —————
135 df['order_size'] = pd.cut(
136     df['sales_amount'],
137     bins    = [0, 5000, 25000, 75000, 999999],
138     labels  = ['Small', 'Medium', 'Large', 'Premium']
139 )
140
141 # — Revenue Contribution % —————
142 total_rev = df[df['status'] == 'Delivered']['sales_amount'].sum()
143 df['revenue_contribution_pct'] = round(
144     df['sales_amount'] / total_rev * 100, 4)
145
146 print("New columns added:")
147 new_cols = ['order_day_name', 'order_quarter', 'order_week',
148             'discount_amount', 'gross_amount', 'profit',
149             'order_size', 'revenue_contribution_pct']
150 print(df[new_cols].head(4).to_string())
151
152
153
154 # STEP 8 – OUTLIER CHECK
155
156 print("\n" + "=" * 55)
157 print("STEP 8 – OUTLIER CHECK (sales_amount)")
158 print("=" * 55)
159
160 Q1  = df['sales_amount'].quantile(0.25)
161 Q3  = df['sales_amount'].quantile(0.75)
162 IQR = Q3 - Q1
163 lower = Q1 - 1.5 * IQR
164 upper = Q3 + 1.5 * IQR
165
166 outliers = df[(df['sales_amount'] < lower) |
167               (df['sales_amount'] > upper)]
168
169 print(f"Q1          : {Q1}")
170 print(f"Q3          : {Q3}")
171 print(f"IQR         : {IQR}")
172 print(f"Lower Bound  : {lower}")
173 print(f"Upper Bound  : {upper}")
174 print(f"Outliers     : {len(outliers)} rows")
175
176 # Note: We keep outliers – they are valid high-value orders
177 print(" Outliers kept – they are valid premium orders")
178
179
180

```

```

181 # STEP 9 – CUSTOMER SEGMENTATION
182
183 print("\n" + "=" * 55)
184 print("STEP 9 – CUSTOMER SEGMENTATION")
185 print("=" * 55)
186
187 customer_spend = df[df['status'] == 'Delivered'].groupby(
188     'customer_name')['sales_amount'].sum()
189
190 def segment(spent):
191     if spent >= 1500000: return 'High Value'
192     elif spent >= 800000: return 'Medium Value'
193     else: return 'Low Value'
194
195 customer_segment = customer_spend.apply(segment).reset_index()
196 customer_segment.columns = ['customer_name', 'customer_segment']
197
198 df = df.merge(customer_segment, on='customer_name', how='left')
199 df['customer_segment'] = df['customer_segment'].fillna('Low Value')
200
201 print(f"Segment distribution:\n"
202       f"{df['customer_segment'].value_counts()}")
203
204
205 # STEP 10 – SPLIT INTO SUB DATASETS
206 print("\n" + "=" * 55)
207 print("STEP 10 – CREATE SUB DATASETS")
208 print("=" * 55)
209
210 # Only delivered orders (for dashboard)
211 df_delivered = df[df['status'] == 'Delivered'].copy()
212
213 # Only 2023 data
214 df_2023 = df[df['order_year'] == 2023].copy()
215
216 # Only 2024 data
217 df_2024 = df[df['order_year'] == 2024].copy()
218
219 print(f"All orders      : {len(df)}")
220 print(f"Delivered only   : {len(df_delivered)}")
221 print(f"2023 orders       : {len(df_2023)}")
222 print(f"2024 orders       : {len(df_2024)}")
223
224
225 # STEP 11 – FINAL CLEANED DATA SUMMARY
226
227 print("\n" + "=" * 55)
228 print("STEP 11 – FINAL DATA SUMMARY")
229 print("=" * 55)
230
231 print(f"Final Shape      : {df.shape}")
232 print(f"Total Columns    : {len(df.columns)}")
233 print(f"\nAll Columns:\n{list(df.columns)}")
234 print(f"\nTotal Revenue     : Rs.{df_delivered['sales_amount'].sum():,.0f}")
235 print(f"Total Orders      : {len(df_delivered)}")
236 print(f"Avg Order Value   : Rs.{df_delivered['sales_amount'].mean():,.0f}")
237 print(f"Total Profit      : Rs.{df_delivered['profit'].sum():,.0f}")
238
239
240 # STEP 12 – SAVE ALL CLEANED FILES

```

```

241
242 print("\n" + "=" * 55)
243 print("STEP 12 – SAVING CLEANED FILES")
244 print("=" * 55)
245
246 os.makedirs("data", exist_ok=True)
247
248 df.to_csv("data/sales_cleaned.csv", index=False)
249 df_delivered.to_csv("data/sales_delivered.csv", index=False)
250 df_2023.to_csv("data/sales_2023.csv", index=False)
251 df_2024.to_csv("data/sales_2024.csv", index=False)
252
253 print(f" data/sales_cleaned.csv      → {df.shape}")
254 print(f" data/sales_delivered.csv   → {df_delivered.shape}")
255 print(f" data/sales_2023.csv        → {df_2023.shape}")
256 print(f" data/sales_2024.csv        → {df_2024.shape}")
257
258 print("\n" + "=" * 55)
259 print(" PHASE 2 COMPLETE!")
260 print("Next Step → Phase 3: EDA with Matplotlib & Seaborn")
261 print("=" * 55)
262
263
264 # PHASE 3 – EXPLORATORY DATA ANALYSIS (EDA)
265
266 import pandas as pd
267 import matplotlib.pyplot as plt
268 import matplotlib.ticker as mticker
269 import seaborn as sns
270 import warnings
271 import os
272
273 warnings.filterwarnings('ignore')
274 os.makedirs("charts", exist_ok=True)
275
276
277
278 sns.set_theme(style="whitegrid")
279 plt.rcParams['figure.figsize'] = (12, 5)
280 plt.rcParams['font.size'] = 11
281
282
283 model =
pd.read_csv(r"C:\Users\Harikesh\OneDrive\Desktop\Sales_Dashboard.csv",
284           parse_dates=["order_date"])
285 df_delivered = model[model['status'] == 'Delivered'].copy()
286
287 print(f"Loaded: {model.shape} | Delivered: {df_delivered.shape}")
288
289
290 #CHART 1 – ORDER STATUS BREAKDOWN (Pie Chart)
291
292 status_counts = df['status'].value_counts()
293
294 plt.figure(figsize=(7, 7))
295 colors = ['#2ecc71', '#f39c12', '#e74c3c']
296 plt.pie(status_counts,
297         labels = status_counts.index,
298         autopct = '%1.1f%%',
299         colors = colors,

```

```

300     startangle= 140,
301     explode = [0.05, 0, 0])
302 plt.title("Order Status Breakdown", fontsize=16, fontweight='bold')
303 plt.tight_layout()
304 plt.savefig("charts/01_order_status.png", dpi=150)
305 plt.show()
306 print(" Chart 1 saved → charts/01_order_status.png")
307
308
309 # CHART 2 – MONTHLY SALES TREND (Line Chart)
310
311 region = (df_delivered
312     .groupby('region')['sales_amount']
313     .sum()
314     .sort_values(ascending=False)
315     .reset_index())
316
317 plt.figure()
318 bars = plt.bar(region['region'], region['sales_amount'],
319     color=['#3498db', '#2ecc71', '#e74c3c',
320     '#f39c12', '#9b59b6'],
321     edgecolor='white', linewidth=0.8)
322
323 # Add value labels on bars
324 for bar in bars:
325     h = bar.get_height()
326     plt.text(bar.get_x() + bar.get_width()/2,
327         h + 30000,
328         f'₹{h/1e5:.1f}L',
329         ha='center', fontsize=10, fontweight='bold')
330
331 plt.title("Total Revenue by Region",
332     fontsize=14, fontweight='bold')
333 plt.xlabel("Region")
334 plt.ylabel("Revenue (Rs.)")
335 plt.gca().yaxis.set_major_formatter(
336     mticker.FuncFormatter(lambda x, _: f'₹{x/1e5:.0f}L'))
337 plt.tight_layout()
338 plt.savefig("charts/03_region_sales.png", dpi=150)
339 plt.show()
340
341
342 # CHART 4 – TOP 10 PRODUCTS (Horizontal Bar)
343
344 products = (df_delivered
345     .groupby('product_name')['sales_amount']
346     .sum()
347     .sort_values(ascending=True)
348     .tail(10))
349
350 plt.figure()
351 bars = plt.barh(products.index, products.values,
352     color='#3498db', edgecolor='white')
353
354 for bar in bars:
355     w = bar.get_width()
356     plt.text(w + 20000, bar.get_y() + bar.get_height()/2,
357         f'₹{w/1e5:.1f}L',
358         va='center', fontsize=9)
359

```

```

360 plt.title("Top 10 Products by Revenue",
361           fontsize=14, fontweight='bold')
362 plt.xlabel("Revenue (Rs.)")
363 plt.tight_layout()
364 plt.savefig("charts/04_top_products.png", dpi=150)
365 plt.show()
366
367
368 # CHART 5 – REVENUE BY CATEGORY (Donut Chart)
369
370 category = (df_delivered
371             .groupby('category')['sales_amount']
372             .sum()
373             .sort_values(ascending=False))
374
375 plt.figure(figsize=(7, 7))
376 colors = ['#3498db', '#2ecc71', '#e74c3c', '#f39c12']
377 wedges, texts, autotexts = plt.pie(
378     category,
379     labels = category.index,
380     autopct = '%1.1f%%',
381     colors = colors,
382     startangle= 90,
383     wedgeprops= dict(width=0.6),    # donut hole
384     pctdistance=0.75
385 )
386 plt.title("Revenue by Category",
387           fontsize=16, fontweight='bold')
388 plt.tight_layout()
389 plt.savefig("charts/05_category_revenue.png", dpi=150)
390 plt.show()
391
392
393 # CHART 6 – TOP 10 CUSTOMERS (Horizontal Bar)
394
395
396 customers = (df_delivered
397             .groupby('customer_name')['sales_amount']
398             .sum()
399             .sort_values(ascending=True)
400             .tail(10))
401
402 plt.figure()
403 plt.barh(customers.index, customers.values,
404          color='#9b59b6', edgecolor='white')
405
406 for i, (name, val) in enumerate(zip(customers.index,
407                                    customers.values)):
408     plt.text(val + 10000, i,
409             f'₹{val/1e5:.1f}L',
410             va='center', fontsize=9)
411
412 plt.title("Top 10 Customers by Revenue",
413           fontsize=14, fontweight='bold')
414 plt.xlabel("Revenue (Rs.)")
415 plt.tight_layout()
416 plt.savefig("charts/06_top_customers.png", dpi=150)
417 plt.show()
418
419

```

```

420 # CHART 7 – YEAR COMPARISON 2023 vs 2024 (Grouped Bar)
421
422 year_month = (df_delivered
423               .groupby(['order_year', 'order_month'])['sales_amount']
424               .sum()
425               .reset_index())
426
427 y2023 = year_month[year_month['order_year'] == 2023].set_index(
428         'order_month')['sales_amount']
429 y2024 = year_month[year_month['order_year'] == 2024].set_index(
430         'order_month')['sales_amount']
431
432 months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
433           'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
434 x = range(1, 13)
435
436 plt.figure()
437 plt.bar([i - 0.2 for i in x], y2023.reindex(x, fill_value=0),
438         width=0.4, label='2023', color='#3498db', alpha=0.85)
439 plt.bar([i + 0.2 for i in x], y2024.reindex(x, fill_value=0),
440         width=0.4, label='2024', color='#e74c3c', alpha=0.85)
441 plt.xticks(list(x), months)
442 plt.title("2023 vs 2024 Monthly Revenue Comparison",
443         fontsize=14, fontweight='bold')
444 plt.xlabel("Month")
445 plt.ylabel("Revenue (Rs.)")
446 plt.legend()
447 plt.gca().yaxis.set_major_formatter(
448     mticker.FuncFormatter(lambda x, _: f'₹{x/1e5:.1f}L'))
449 plt.tight_layout()
450 plt.savefig("charts/07_year_comparison.png", dpi=150)
451 plt.show()
452
453
454 # CHART 8 – SALES DISTRIBUTION (Histogram)
455
456 plt.figure()
457 plt.hist(df_delivered['sales_amount'],
458         bins=30, color='#2980b9',
459         edgecolor='white', alpha=0.8)
460 plt.axvline(df_delivered['sales_amount'].mean(),
461         color='red', linestyle='--',
462         label=f"Mean: ₹{df_delivered['sales_amount'].mean():.0f}")
463 plt.axvline(df_delivered['sales_amount'].median(),
464         color='green', linestyle='--',
465         label=f"Median: ₹{df_delivered['sales_amount'].median():.0f}")
466 plt.title("Sales Amount Distribution",
467         fontsize=14, fontweight='bold')
468 plt.xlabel("Sales Amount (Rs.)")
469 plt.ylabel("Frequency")
470 plt.legend()
471 plt.tight_layout()
472 plt.savefig("charts/08_sales_distribution.png", dpi=150)
473 plt.show()
474 print(" Chart 8 saved → charts/08_sales_distribution.png")
475
476
477
478 # CHART 9 – DISCOUNT vs SALES (Box Plot)
479

```



```

480 plt.figure()
481 sns.boxplot(data=df_delivered,
482             x='discount_pct',
483             y='sales_amount',
484             palette='Blues')
485 plt.title("Sales Amount by Discount %",
486           fontsize=14, fontweight='bold')
487 plt.xlabel("Discount %")
488 plt.ylabel("Sales Amount (Rs.)")
489 plt.gca().yaxis.set_major_formatter(
490     mticker.FuncFormatter(lambda x, _: f'₹{x/1e3:.0f}K'))
491 plt.tight_layout()
492 plt.savefig("charts/09_discount_boxplot.png", dpi=150)
493 plt.show()
494
495
496 # PRINT KEY INSIGHTS
497
498
499 print("\n" + "=" * 55)
500 print("KEY BUSINESS INSIGHTS FROM EDA")
501 print("=" * 55)
502
503 top_region    = df_delivered.groupby('region')['sales_amount'].sum().idxmax()
504 top_product   = df_delivered.groupby('product_name')['sales_amount'].sum().idxmax()
505 top_customer  = df_delivered.groupby('customer_name')['sales_amount'].sum().idxmax()
506 top_category  = df_delivered.groupby('category')['sales_amount'].sum().idxmax()
507 top_month     = df_delivered.groupby('year_month')['sales_amount'].sum().idxmax()
508
509 print(f"Total Revenue    : Rs.{df_delivered['sales_amount'].sum():,.0f}")
510 print(f"Total Orders     : {len(df_delivered)}")
511 print(f"Avg Order Value   : Rs.{df_delivered['sales_amount'].mean():,.0f}")
512 print(f"Top Region        : {top_region}")
513 print(f"Top Product       : {top_product}")
514 print(f"Top Customer      : {top_customer}")
515 print(f"Top Category      : {top_category}")
516 print(f"Best Month        : {top_month}")
517 print(f"Avg Discount     : {df_delivered['discount_pct'].mean():.1f}%")
518
519
520 if 'profit' in df_delivered.columns:
521     print(f"Total Profit      : Rs.{df_delivered['profit'].sum():,.0f}")
522 else:
523     profit = df_delivered['sales_amount'].sum() * 0.30
524     print(f"Total Profit      : Rs.{profit:,.0f} (estimated 30%)")
525
526 print("\n" + "=" * 55)
527 print(" PHASE 3 COMPLETE!")
528 print("Next Step → Phase 4: Sales Forecasting with ML")
529 print("=" * 55)
530
531
532
533 # PHASE 4 – SALES FORECASTING WITH MACHINE LEARNING
534
535
536 import pandas as pd
537 import numpy as np
538 import matplotlib.pyplot as plt
539 import matplotlib.ticker as mticker

```

```

540 import warnings
541 import os
542
543 from sklearn.linear_model import LinearRegression
544 from sklearn.metrics import mean_absolute_error, r2_score
545 from sklearn.model_selection import train_test_split
546 from sklearn.preprocessing import LabelEncoder
547
548 warnings.filterwarnings('ignore')
549 os.makedirs("charts", exist_ok=True)
550 os.makedirs("models", exist_ok=True)
551 os.makedirs("data", exist_ok=True)
552
553
554 # LOAD DATA
555
556 print("=" * 55)
557 print("STEP 1 – LOAD DATA")
558 print("=" * 55)
559
560 model = pd.read_csv(r"C:\Users\Harikesh\OneDrive\Desktop\Sales_Dashboard.csv",
561                    parse_dates=["order_date"])
562 df_delivered = df[df['status'] == 'Delivered'].copy()
563
564 print(f"Total rows      : {len(df)}")
565 print(f"Delivered rows   : {len(df_delivered)}")
566
567
568 print("\n" + "=" * 55)
569 print("STEP 2 – PREPARE MONTHLY DATA")
570 print("=" * 55)
571
572 # Group by month → get total revenue per month
573 monthly = (df_delivered
574            .groupby('year_month')['sales_amount']
575            .sum()
576            .reset_index())
577 monthly.columns = ['year_month', 'revenue']
578
579 # Add month number (1 to 24) as feature for ML
580 monthly['month_number'] = range(1, len(monthly) + 1)
581
582 print(f"Monthly records : {len(monthly)}")
583 print(monthly.to_string(index=False))
584
585
586 print("\n" + "=" * 55)
587 print("STEP 3 – FEATURES & TARGET SETUP")
588 print("=" * 55)
589
590 X = monthly[['month_number']] # Feature → month number
591 y = monthly['revenue']       # Target → revenue
592
593 # Split into train and test
594 X_train, X_test, y_train, y_test = train_test_split(
595     X, y, test_size=0.2, shuffle=False
596 )
597
598 print(f"X_train shape : {X_train.shape}")
599 print(f"X_test shape  : {X_test.shape}")

```

```

600 print(f"y_train shape : {y_train.shape}")
601 print(f"y_test shape : {y_test.shape}")
602
603
604 print("\n" + "=" * 55)
605 print("STEP 4 – TRAIN LINEAR REGRESSION MODEL")
606 print("=" * 55)
607
608 model = LinearRegression()
609 model.fit(X_train, y_train)
610
611 print(f"Model Coefficient : {model.coef_[0]:,.2f}")
612 print(f"Model Intercept : {model.intercept_:,.2f}")
613 print(f"\nMeaning → Revenue increases by "
614       f"Rs.{model.coef_[0]:,.0f} every month")
615
616 print("\n" + "=" * 55)
617 print("STEP 5 – MODEL EVALUATION")
618 print("=" * 55)
619
620 y_pred = model.predict(X_test)
621
622 mae = mean_absolute_error(y_test, y_pred)
623 r2 = r2_score(y_test, y_pred)
624
625 print(f"MAE (Mean Absolute Error) : Rs.{mae:,.0f}")
626 print(f"R² Score : {r2:,.4f}")
627 print()
628
629
630 # Interpret scores
631 if r2 >= 0.8:
632     print(" R² Score is GOOD – model explains trend well")
633 elif r2 >= 0.6:
634     print(" R² Score is MODERATE – acceptable for trend")
635 else:
636     print(" R² Score is LOW – trend is non-linear")
637
638 print()
639 print("Actual vs Predicted (Test Set):")
640 comparison = pd.DataFrame({
641     'Month' : X_test['month_number'].values,
642     'Actual' : y_test.values,
643     'Predicted': y_pred.round(0)
644 })
645 print(comparison.to_string(index=False))
646
647
648 print("\n" + "=" * 55)
649 print("STEP 6 – FORECAST NEXT 6 MONTHS")
650 print("=" * 55)
651
652 # Generate next 6 month numbers
653 last_month = monthly['month_number'].max()
654 future_months = pd.DataFrame({
655     'month_number': range(last_month + 1,
656                          last_month + 7)
657 })
658
659 # Generate future month labels (2025-01 to 2025-06)

```

```

660 future_labels = pd.date_range(
661     start='2025-01-01', periods=6, freq='MS'
662 ).strftime('%Y-%m').tolist()
663
664 future_months['year_month'] = future_labels
665 future_months['predicted_revenue'] = model.predict(
666     future_months[['month_number']]).round(0)
667
668 print("Forecast for Next 6 Months:")
669 print(future_months[['year_month',
670     'predicted_revenue']].to_string(index=False))
671 print(f"\nTotal Forecasted Revenue (6 months): "
672     f"Rs.{future_months['predicted_revenue'].sum():.0f}")
673
674 print("\n" + "=" * 55)
675 print("STEP 7 – CREATING FORECAST CHARTS")
676 print("=" * 55)
677
678 # Predict on full dataset
679 monthly['predicted'] = model.predict(
680     monthly[['month_number']]).round(0)
681
682 plt.figure(figsize=(14, 6))
683 plt.plot(monthly['year_month'],
684     monthly['revenue'],
685     marker='o', color='#2980b9',
686     linewidth=2.5, label='Actual Revenue',
687     markersize=5)
688
689 plt.plot(monthly['year_month'],
690     monthly['predicted'],
691     linestyle='--', color='#e74c3c',
692     linewidth=2, label='Trend Line')
693
694 plt.xticks(rotation=45, ha='right')
695 plt.title("Actual Revenue vs Trend Line",
696     fontsize=14, fontweight='bold')
697 plt.xlabel("Month")
698 plt.ylabel("Revenue (Rs.)")
699 plt.legend()
700 plt.gca().yaxis.set_major_formatter(
701     mticker.FuncFormatter(
702         lambda x, _: f'Rs.{x/1e5:.1f}L'))
703 plt.tight_layout()
704 plt.savefig("charts/11_actual_vs_trend.png", dpi=150)
705 plt.show()
706
707
708 all_months = list(monthly['year_month']) + future_labels
709 actual_rev = list(monthly['revenue']) + [None] * 6
710 forecast_rev = [None] * len(monthly) + \
711     list(future_months['predicted_revenue'])
712
713 plt.figure(figsize=(16, 6))
714 plt.plot(all_months[:len(monthly)],
715     actual_rev[:len(monthly)],
716     marker='o', color='#2980b9',
717     linewidth=2.5, label='Actual Revenue',
718     markersize=5)
719

```

```

720 plt.plot(all_months[len(monthly)-1:],
721           [actual_rev[len(monthly)-1]] + forecast_rev[len(monthly):],
722           marker='s', color='#e74c3c',
723           linewidth=2.5, linestyle='--',
724           label='Forecasted Revenue',
725           markersize=7, markerfacecolor='white',
726           markeredgewidth=2)
727
728 # Shade forecast region
729 plt.axvspan(all_months[len(monthly)-1],
730             all_months[-1],
731             alpha=0.08, color='#e74c3c',
732             label='Forecast Period')
733
734 plt.xticks(rotation=45, ha='right')
735 plt.title("Sales Forecast – Next 6 Months (2025)",
736           fontsize=14, fontweight='bold')
737 plt.xlabel("Month")
738 plt.ylabel("Revenue (Rs.)")
739 plt.legend()
740 plt.gca().yaxis.set_major_formatter(
741     mticker.FuncFormatter(
742         lambda x, _: f'Rs.{x/1e5:.1f}L'))
743 plt.tight_layout()
744 plt.savefig("charts/12_forecast_6months.png", dpi=150)
745 plt.show()
746
747
748 print("\n" + "=" * 55)
749 print("STEP 9 – REGION WISE FORECAST")
750 print("=" * 55)
751
752 region_forecast = []
753
754 for region in df_delivered['region'].unique():
755     df_region = (df_delivered[df_delivered['region'] == region]
756                 .groupby('year_month')['sales_amount']
757                 .sum()
758                 .reset_index())
759     df_region['month_number'] = range(1, len(df_region) + 1)
760
761     reg_model = LinearRegression()
762     reg_model.fit(df_region[['month_number']],
763                  df_region['sales_amount'])
764
765     next_month = df_region['month_number'].max() + 1
766     forecast = reg_model.predict([[next_month]])[0]
767
768     region_forecast.append({
769         'region': region,
770         'next_month_forecast': round(forecast, 0)
771     })
772
773 region_fc_df = pd.DataFrame(region_forecast).sort_values(
774     'next_month_forecast', ascending=False)
775
776 print("Region-wise Forecast for Next Month:")
777 print(region_fc_df.to_string(index=False))
778
779 # Plot region forecast

```

```

780 plt.figure(figsize=(10, 5))
781 colors = ['#3498db', '#2ecc71', '#e74c3c', '#f39c12', '#9b59b6']
782 bars = plt.bar(region_fc_df['region'],
783                 region_fc_df['next_month_forecast'],
784                 color=colors, edgecolor='white')
785
786 for bar in bars:
787     h = bar.get_height()
788     plt.text(bar.get_x() + bar.get_width() / 2,
789             h + 1000,
790             f'Rs.{h/1e5:.2f}L',
791             ha='center', fontsize=10,
792             fontweight='bold')
793
794 plt.title("Region-wise Revenue Forecast (Next Month)",
795           fontsize=14, fontweight='bold')
796 plt.xlabel("Region")
797 plt.ylabel("Forecasted Revenue (Rs.)")
798 plt.tight_layout()
799 plt.savefig("charts/13_region_forecast.png", dpi=150)
800 plt.show()
801
802
803 print("\n" + "=" * 55)
804 print("STEP 10 – SAVE RESULTS")
805 print("=" * 55)
806
807 # Save forecast CSV
808 future_months.to_csv("data/forecast_results.csv", index=False)
809 print(" data/forecast_results.csv saved")
810
811 # Save monthly with predictions
812 monthly.to_csv("data/monthly_with_predictions.csv", index=False)
813 print(" data/monthly_with_predictions.csv saved")
814
815
816 print("\n" + "=" * 55)
817 print("PHASE 4 COMPLETE – SUMMARY")
818 print("=" * 55)
819 print(f"Model Used           : Linear Regression")
820 print(f"MAE                   : Rs.{mae:,.0f}")
821 print(f"R2 Score             : {r2:.4f}")
822 print(f"Training Months        : {len(X_train)}")
823 print(f"Test Months            : {len(X_test)}")
824 print(f"\n6-Month Forecast:")
825 print(future_months[['year_month',
826                     'predicted_revenue']].to_string(index=False))
827 print("\n" + "=" * 55)
828
829

```